Handover Document

Live-Rate Jewellery Configurator, Cart & Checkout

Contents

Everything needed to run, test, demo and maintain the Live-Rate Jewellery Configurator.

---

1. What you are receiving

Item	Location
Source code (React client, Node server, shared pricing engine)	client/, server/, shared/
Automated tests (29)	server/test/
Sample data (4 products, 3 metal rates)	server/src/seed.js (loaded automatically)
One-command setup	docker-compose.yml, Dockerfile
README (assumptions, rounding, future work)	README.md
Business requirements	docs/BRD.md
Technical documentation (flows, API)	docs/DOCUMENTATION.md
This handover	docs/HANDOVER.md

2. Prerequisites

Recommended: Docker Desktop — nothing else needs installing (Node and PostgreSQL run inside containers).

OS	Install
Windows 10/11	Docker Desktop + an up-to-date WSL (see Troubleshooting)
macOS	Docker Desktop
Linux	Docker Engine + Compose plugin

Check it is ready:

```
docker --version
docker compose version
```

Docker Desktop must show **Engine running** before you continue.

If Docker cannot run on your machine (see Troubleshooting), follow Running without Docker instead — it needs only Node.js and PostgreSQL.

Free port needed: **3000** (app). PostgreSQL is not exposed outside Docker.

3. Run the application

From the project folder (the one containing docker-compose.yml):

```
docker compose up --build
```

- First run takes 2–5 minutes and needs internet access (downloads the node and postgres images and npm packages, builds the client). Later starts take seconds.
- If the app was previously run **without Docker** on the same machine, stop those PowerShell windows first (Ctrl + C) so port 3000 is free. A local PostgreSQL install does not conflict: the Docker database is not exposed on port 5432.
- Ready when the log shows: Jewellery store on http://localhost:3000
- Open **http://localhost:3000**

What happens on start-up:

1. PostgreSQL starts; a jewellery_test database is also created for tests (the tests also create it themselves if missing).
2. The app starts only once the database passes its health check, then waits for it, applies the schema, and seeds sample data **if the products table is empty**.
3. The live-rate ticker starts (rates move every 60 s).
4. The API and the built React app are served on port 3000. Both containers report **healthy** in docker compose ps and restart automatically if they stop unexpectedly.

Stop: press Ctrl + C, or from another terminal docker compose down.

Run in the background instead:

```
docker compose up --build -d
docker compose logs -f app      # follow logs
docker compose down             # stop
```

4. Run the tests

In a second terminal, same folder:

```
docker compose run --rm --build test
```

Expected ending:

```
# tests 29
# pass 29
# fail 0
```

Tests use their own jewellery_test database and reset it themselves, so they never touch demo data and can be run while the app is running.

5. Demo script

A 10-minute lock is slow to demo. Restart with a 30-second lock and frozen rates:

Windows PowerShell

```
docker compose down
$env:PRICE_LOCK_SECONDS=30; $env:RATE_TICKER="off"; docker compose up --build
```

macOS / Linux

```
docker compose down
PRICE_LOCK_SECONDS=30 RATE_TICKER=off docker compose up --build
```

(Close the terminal afterwards, or run `Remove-Item Env:PRICE_LOCK_SECONDS, Env:RATE_TICKER` in PowerShell, to go back to normal.)

Open the shop in one tab and **http://localhost:3000/admin** (token `admin`) in another.

#	Scenario	Steps	Expected
1	Price breakdown	Open *Aarna Solitaire Ring*; switch 22K ↔️ 18K and sizes	Price and every breakdown line update instantly; countdown visible
2	Add to cart	Click **Add to cart**	"Added to your cart at this price"; cart badge shows 1
3	Price rise	In admin, raise gold by ~₹1,000; open the cart; wait for the countdown	Item shows old price struck through, new price, **Accept** button; checkout disabled; summary says "1 item needs price confirmation"
4	Accept	Click **Accept**	Item confirmed, fresh countdown, checkout enabled
5	Price drop	Wait for the next expiry after lowering gold in admin	"Price dropped … lower price applied", old price struck through
6	Unchanged	Let a lock expire without changing the rate	"Price re-checked … no change"
7	Checkout	Click **Checkout**	Order confirmation; charged amount = total shown
8	Out of stock	Add *Meher Temple Necklace* (1 in stock) to cart. In a **private/incognito window**, open it and click **Buy now**. Return to the first window and wait for the countdown	Incognito order succeeds; first cart marks the necklace out of stock and blocks checkout
9	Buy now, expired price	Open a product, change the rate in admin, wait for the countdown	Product page shows the price change; for a rise, Buy now stays disabled until accepted
10	Live rates	Restart without RATE_TICKER=off	Header ticker flashes red/green as rates move; locked prices don't change
11	Mobile	Browser dev tools →	Layouts stack to one

#	Scenario	Steps	Expected
		device toolbar (iPhone size)	column, all actions reachable

Reset stock after the demo: see section 7.

6. Configuration

Set as environment variables before `docker compose up` (see demo script for syntax).

Variable	Default	Purpose
PRICE_LOCK_SECONDS	*(empty = 600, i.e. 10 min)*	Length of every price lock
RATE_TICKER	on	off stops automatic rate changes
RATE_TICK_MS	60000	Interval between automatic rate changes (ms)
ADMIN_TOKEN	admin	Token for /admin and admin API
DATABASE_URL	set by compose	PostgreSQL connection (app)
TEST_DATABASE_URL	set by compose	PostgreSQL connection (tests)
APP_PORT	3000	Port on the host machine (open http://localhost:APP_PORT)
PORT	3000	HTTP port inside the container (no need to change)

To use a different host port (e.g. if 3000 is busy), set `APP_PORT` — for example `$env:APP_PORT=8080; docker compose up --build` (PowerShell) or `APP_PORT=8080 docker compose up --build` (macOS/Linux) — and open http://localhost:8080.

7. Resetting and managing data

Data lives in a Docker volume (`pgdata`) and survives restarts. Orders reduce stock permanently.

Full reset to sample data:

```
docker compose down -v
docker compose up --build
```

`-v` deletes the volume; seed data is reloaded on the next start.

Inspect the database:

```
docker compose exec db psql -U postgres -d jewellery
```

Useful queries:

```
SELECT slug, stock FROM products;
SELECT DISTINCT ON (metal) metal, rate_paise_per_gram/100.0 AS rupees FROM metal_rates ORDER BY metal,
id DESC;
SELECT id, total_paise/100.0 AS rupees, source, created_at FROM orders ORDER BY id DESC;
UPDATE products SET stock = 1 WHERE slug = 'meher-temple-necklace';   -- restock
```

8. Running without Docker

Use this when Docker cannot run (for example, virtualization is disabled in the BIOS on a managed laptop). Needs **Node.js 22 or newer** (built on Node 22; also runs on Node 24) and **PostgreSQL 14 or newer** (built on PostgreSQL 16; also runs on PostgreSQL 18).

In this mode the app runs as **two processes**: the API on port 3000 and the website on port 5173.

8.1 Windows (PowerShell), step by step

1. Install the prerequisites

- Node.js LTS from https://nodejs.org (accept the defaults).
- PostgreSQL from https://www.postgresql.org/download/windows/. Tick **PostgreSQL Server** and **Command Line Tools**, keep port **5432**, and remember the password you set for the `postgres` user. **Stack Builder** at the end is not needed: click **Cancel**.

2. Allow npm to run in PowerShell (one time, no admin needed)

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Answer **Y**. (Alternative without changing the setting: type `npm.cmd` instead of `npm` in every command.)

3. Create the two databases. This finds `psql` whatever PostgreSQL version is installed:

```
$psql = (Get-ChildItem "C:\Program Files\PostgreSQL\*\bin\psql.exe" | Select-Object -First 1).FullName; $psql
& $psql -U postgres -c "CREATE DATABASE jewellery;"
& $psql -U postgres -c "CREATE DATABASE jewellery_test;"
```

Enter the PostgreSQL password when asked; each command prints `CREATE DATABASE`.

4. Go to the project folder and install packages

```
cd C:\Users\<you>\Downloads\jewellery-configurator
npm run install:all
```

Lines about *vulnerabilities* (development tools only) and *esbuild install scripts* can be ignored. Do not run `npm audit fix --force`.

5. Start the API — window 1 (replace `YOURPASSWORD` with the real PostgreSQL password)

```
$env:DATABASE_URL="postgres://postgres:YOURPASSWORD@localhost:5432/jewellery"; npm run dev:server
```

Ready when it shows `Jewellery store on http://localhost:3000`. Leave the window open.

6. Start the website — window 2

```
cd C:\Users\<you>\Downloads\jewellery-configurator
npm run dev:client
```

Open **http://localhost:5173**.

> Opening http://localhost:3000 in this mode shows **"Cannot GET /"**. That is expected: port 3000 is only the API (try http://localhost:3000/api/products). To serve the website from port 3000 as well, run `npm --prefix client run build` once and restart window 1.

7. Run the tests — window 3

```
cd C:\Users\<you>\Downloads\jewellery-configurator
$env:TEST_DATABASE_URL="postgres://postgres:YOURPASSWORD@localhost:5432/jewellery_test"; npm run test:local
```

Expected ending: `# pass 29`, `# fail 0`.

8. 30-second lock for the demo — stop window 1 with `Ctrl + C`, then:

```
$env:DATABASE_URL="postgres://postgres:YOURPASSWORD@localhost:5432/jewellery";
$env:PRICE_LOCK_SECONDS=30; $env:RATE_TICKER="off"; npm run dev:server
```

Reset sample data (no Docker):

```
& $psql -U postgres -c "DROP DATABASE jewellery;"
& $psql -U postgres -c "CREATE DATABASE jewellery;"
```

Then restart window 1; the schema and sample data are recreated automatically.

8.2 macOS / Linux

```
createdb -U postgres jewellery
createdb -U postgres jewellery_test
npm run install:all
DATABASE_URL=postgres://postgres:postgres@localhost:5432/jewellery npm run dev:server     # terminal 1
npm run dev:client                                                  # terminal 2 → http://localhost:5173
TEST_DATABASE_URL=postgres://postgres:postgres@localhost:5432/jewellery_test npm run test:local
```

Adjust the user and password in the URLs to match your PostgreSQL install.

> **Password tip:** if the PostgreSQL password contains `@ # % / : ?`, URL-encode it in the connection string (for example `@` → `%40`, `#` → `%23`).

9. Troubleshooting

9.1 Windows: Docker Desktop errors

"WSL needs updating" Open **PowerShell as administrator**:

```
wsl --update
```

If that prints a usage/help screen (older built-in WSL), use:

```
wsl --install --web-download --no-distribution
```

Restart the PC, then check `wsl --version` prints a version. If both fail, install the latest `.x64.msi` from https://github.com/microsoft/WSL/releases/latest and restart.

If Docker reports virtualization is disabled, run (admin PowerShell) and restart:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

"Virtualization is disabled in firmware" — hardware virtualization must be turned on in the BIOS/UEFI:

1. Windows **Settings → System → Recovery → Advanced startup → Restart now**, then **Troubleshoot → Advanced options → UEFI Firmware Settings → Restart**. (Or press the BIOS key while the PC starts: usually F2, F10, F1 or Del.)

2. Find **Intel Virtualization Technology** / **Intel VT-x** or **SVM Mode** / **AMD-V** (under *Advanced*, *CPU Configuration* or *Security*) and set it to **Enabled**.
3. Save and exit (usually F10).
4. Check **Task Manager → Performance → CPU → Virtualization: Enabled**, then start Docker Desktop.

On a managed work/school laptop the BIOS may be locked; use **section 8 (Running without Docker)** instead.

9.2 Other problems

Symptom	Cause	Fix
Cannot connect to the Docker daemon / error during connect	Docker Desktop not running	Start it; wait for *Engine running*
port is already allocated / address already in use	Something else uses port 3000 (e.g. the non-Docker dev server still running)	Stop it, or start with APP_PORT=8080 (section 6)
no configuration file provided	Terminal is in the wrong folder	cd into the folder with docker-compose.yml
Page shows old stock / old data	Data persists in the volume	docker compose down -v then start again
Fonts look plain	No internet access to Google Fonts	Cosmetic only; system fonts are used
Countdown shows 10 min even with PRICE_LOCK_SECONDS	Variable not set in the same terminal	Set it in the same command (section 5) and rebuild
Tests fail with connection refused	Database not up yet / not using compose	Use docker compose run --rm --build test; without Docker, check the PostgreSQL service is running
Virtualization is disabled in firmware	Hardware virtualization off in BIOS	Enable Intel VT-x / AMD-V (SVM) in BIOS (section 9.1), or run without Docker (section 8)
password authentication failed	Wrong PostgreSQL password in the URL	Use the password you set during install in DATABASE_URL / TEST_DATABASE_URL
Code changes not visible	Image not rebuilt	Always use --build
Without Docker: npm.ps1 cannot be loaded because running scripts is disabled	PowerShell execution policy	Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned, or use npm.cmd
Without Docker: 'C:\Program Files\PostgreSQL\16\bin\psql.exe' is not recognized	A different PostgreSQL version is installed (e.g. 18)	Use the $psql = (Get-ChildItem …) command in section 8.1, which finds any version
Without Docker: Cannot GET / at localhost:3000	Port 3000 is the API only in this mode	Open http://localhost:5173 (after npm run dev:client), or build the client (section 8.1, step 6)
Without Docker: password authentication failed for user "postgres"	Wrong password in DATABASE_URL (e.g. the placeholder left in)	Use the password set when installing PostgreSQL
Without Docker: database	Step 3 not done	Create the databases (section 8.1,

Symptom	Cause	Fix
"jewellery" does not exist		step 3)
Without Docker: ECONNREFUSED 127.0.0.1:5432	PostgreSQL service not running	Start it: Win + R → services.msc → *postgresql-x64-NN* → Start
Without Docker: Missing script: "dev:serve"	Typo	The script is dev:server
Without Docker: website fails to start with an **esbuild** error	Newer npm skipped esbuild's install script	cd client; npm install-scripts approve esbuild; npm rebuild esbuild; cd ..
Without Docker: port 5173 or 3000 already in use	An earlier window is still running	Close the old PowerShell window or press Ctrl + C in it

10. Project structure

```
jewellery-configurator/
├── docker-compose.yml       db + app + test services
├── Dockerfile               multi-stage: client build → test image → runtime image
├── docker/initdb.sql        creates jewellery_test database
├── package.json             convenience scripts
├── README.md
├── docs/                    BRD, technical documentation, handover
├── shared/
│   └── pricing.js           pricing engine (used by server AND client)
├── server/
│   ├── src/
│   │   ├── index.js         start-up: wait for DB, migrate, seed, ticker, listen
│   │   ├── app.js           Express routes + error handler
│   │   ├── config.js        lock length
│   │   ├── clock.js         injectable clock (for tests)
│   │   ├── db.js            pool, migrate, withTx
│   │   ├── errors.js        AppError
│   │   ├── schema.sql       tables
│   │   ├── seed.js          sample products and rates
│   │   └── services/
│   │       ├── rates.js     live rates, ticker, SSE events
│   │       ├── products.js  products, admin rules
│   │       ├── locks.js     price locks, quoteFromLock
│   │       ├── cart.js      re-pricing state machine, accept, checkout
│   │       └── orders.js    commitOrder (stock), buy now, get order
│   └── test/                4 test files + helpers
└── client/
    ├── index.html, vite.config.js
    └── src/
        ├── main.jsx, api.js, hooks.js, components.jsx, styles.css
        └── pages/           Catalog, Product, Cart, Order, Admin
```

11. Common maintenance tasks

Task	Where / how
Change the lock length permanently	LOCK_MINUTES in shared/pricing.js
Change GST rate	GST_BPS in shared/pricing.js (300 = 3%); update hand-calculated tests
Add a product	Add an entry to SEED_PRODUCTS in server/src/seed.js, then docker compose down -v &&

Task	Where / how
	docker compose up --build (or insert a row in products)
Change making / wastage rules	/admin page, or PUT /api/admin/products/:id/pricing-rules
Set a metal rate	/admin page, or PUT /api/admin/rates/:metal
Connect a real rate feed	Replace startRateTicker in server/src/services/rates.js with a poller that calls setRate(pool, metal, paise)
Add a new metal	Allow it in the CHECK constraints in schema.sql, METALS in rates.js, and seed a rate
Change how a price is calculated	Only shared/pricing.js — client and server pick it up together. Add a hand-calculated test
Add an API endpoint	Route in server/src/app.js, logic in a services/ file, throw AppError for expected failures

Rule of thumb: anything that takes stock must go through commitOrder in orders.js.

12. Known limitations

- Stock is reserved only at order time, not when added to cart.
- No idempotency key on checkout (a double-click is safe; a network retry after a successful order is not de-duplicated).
- No payment gateway, customer accounts or admin login (admin uses a shared token).
- Rates are simulated, not from a market feed.
- GST is a flat 3% as per the brief (real-world split is 3% goods / 5% making).
- Product page adds quantity 1 at a time.

13. Submitting to GitHub

1. Create a new, empty repository on GitHub (no README, no .gitignore).
2. In the project folder:

```
git remote add origin https://github.com/<your-username>/<repo-name>.git
  git branch -M main
  git push -u origin main
```

3. If it asks for a password, use a GitHub **personal access token** (Settings → Developer settings → Personal access tokens), or sign in with GitHub Desktop.
4. Check the repo page shows README.md, docs/ and the diagrams render.

node_modules and build output are excluded by .gitignore.